

Cryptography Lib Lab Project Report

Path 2: AES vs DES Algorithm Comparison

Done By: Marwa Hussein Ahmed Ali

Course / Lab: Cryptography / Information Security Lab

Department: Computers and Systems Department

Supervised by: Dr. Ahmed Hussein

Academic Term: Second Semester, 2026

1. Project Overview

This project is a practical cryptography application based on **Path 2: Algorithm Comparison**. The selected scenario is **Secure Student Records**. The program reads a real-format CSV file named `student_records.csv`, encrypts the same file using **AES** and **DES**, saves the encrypted outputs, decrypts them again, and verifies that the recovered files match the original file.

The input file contains fake student records such as student ID, name, email, course, and grade. Fake data is used to avoid submitting real private personal information while still using realistic structured data.

2. Algorithms Used

This project uses **AES-256-GCM** and **DES-CBC**.

AES-256-GCM is a modern and recommended symmetric encryption algorithm. It uses a **256-bit key** and a **128-bit block size**. AES-GCM does not need padding. It produces ciphertext, a nonce, and an authentication tag. The nonce is needed during decryption, and the tag is used for integrity checking.

DES-CBC is an older encryption algorithm. It uses a **56-bit effective key** and a **64-bit block size**. DES is weak compared to AES, so it is used only for educational comparison. DES-CBC needs an IV and PKCS#7 padding because DES works on fixed-size blocks.

Feature	AES-256-GCM	DES-CBC
Key Size	256 bits	56 effective bits
Block Size	128 bits	64 bits
Mode	GCM	CBC
IV / Nonce	Nonce	IV
Padding	Not needed	PKCS#7

Feature	AES-256-GCM	DES-CBC
Security	Modern and recommended	Weak, educational only

3. Project Files and Flow

The project contains main.py, crypto_utils.py, gui.py, requirements.txt, a data folder, and an output folder.

main.py displays the terminal menu and runs the full AES vs DES comparison.

crypto_utils.py contains encryption, decryption, Base64, hashing, and verification functions.

gui.py contains a simple Tkinter GUI as an extra feature.

requirements.txt contains the required library: pycryptodome.

The data folder contains the original CSV file, and the output folder contains encrypted files, decrypted files, and key files.

The program flow is: load the original CSV file, generate AES and DES keys, encrypt the file using AES, encrypt the same file using DES, save encrypted outputs, decrypt both outputs, compare the decrypted files with the original file using SHA-256, and print the final verification result.

The encrypted outputs are saved as student_records_aes_encrypted.json and student_records_des_encrypted.json. The decrypted outputs are saved as student_records_aes_decrypted.csv and student_records_des_decrypted.csv.

Base64 is used because encrypted data is binary and may not be readable directly, so it is stored as text inside JSON files.

4. Results and Verification

The program successfully encrypted and decrypted the CSV file using both AES and DES. In the sample run, AES produced a ciphertext size of **351 bytes**, while DES produced a ciphertext size of **352 bytes**. DES output is slightly larger because DES-CBC uses padding, while AES-GCM does not require padding.

The program also prints encryption and decryption execution times for both algorithms. These values may change depending on the computer and each run.

To prove that decryption worked correctly, the program uses **SHA-256 verification**. It calculates the SHA-256 hash of the original file and compares it with the hash of each decrypted file. If the hashes are identical, the files are exactly the same.

The final result was:

AES verification result: SUCCESS - decrypted file matches original.

DES verification result: SUCCESS - decrypted file matches original.

This proves that both decrypted files matched the original CSV file exactly.

5. Extra Features

The project includes extra features: a simple terminal menu, SHA-256 verification, Base64 encrypted output, execution-time comparison between AES and DES, and a simple Tkinter GUI.

6. Security Understanding Questions

I chose AES and DES because the project follows **Path 2: Algorithm Comparison**. AES is modern and secure, while DES is weak and outdated. Comparing them shows why AES is recommended for modern security.

The key is used to encrypt and decrypt the file. AES uses a 256-bit key, while DES uses a 56-bit effective key.

AES-GCM uses a nonce, and DES-CBC uses an IV. They are needed during decryption and help make encryption less predictable.

The encrypted output looks completely different from the original CSV file. Instead of readable student records, it appears as Base64 ciphertext inside JSON files.

Decryption was proven using SHA-256 hashing. The original file hash matched the decrypted file hashes, which proves that the recovered files are identical to the original file.

One security limitation is that the key files are saved locally in the output folder. In a real system, keys should be stored using a secure key management system. Another limitation is that DES is weak and should not be used for real security.

7. Conclusion

This project successfully demonstrates encryption and decryption using a trusted cryptography library. A real-format CSV file was encrypted using AES and DES, saved as encrypted JSON output, decrypted back into CSV files, and verified using SHA-256 hashing.

The comparison shows that AES is stronger and more secure than DES because AES has a larger key size and is recommended for modern applications. DES is used only for educational comparison.